AURA-HOI: Exact High-Occupancy Itemset Mining on Dense Databases with a Length-Capped Residual Bound

Theory, Mathematical Dominance, Systems Architecture & Empirical Audit

Q. Van

Journal of Intelligent Information Systems (JIIS)
Springer Nature

October 2026

Table of Contents

1. Motivation & Problem Formulation
2. The Length-Capped Residual Bound
3. Algorithmic Architecture & Systems Design
4. Empirical Evaluation & Experimental Audit
5. Conclusion & Future Work

Context: Why Frequent Itemset Mining (FIM) Fails

- **Frequent Itemset Mining (FIM):** Measures frequency across transactions ($\text{supp}(X) \geq \theta$).
- **The Dilution Paradox in Large/Dense Baskets:**
  - A pair $\{A, B\}$ appearing in $10,000$ transactions of size 500 has high support $(10,000)$.
  - Yet, $\{A, B\}$ represents a mere $\frac{2}{500} = 0.4\%$ of each basket content!
  - Downstream applications (recommendation, bundle pricing, clinical diagnosis) mistakenly treat dilute artifacts as core customer behaviors.
- **High-Occupancy Itemsets (HOI) [Deng, 2020]:**
  - Constrains patterns to physically **dominate** the transactions containing them.
  - Occupancy balances recurrence with within-transaction density.

Core Objective

Mine all item combinations whose average relative contribution across the transaction database meets or exceeds a user-defined threshold α.

Formal Problem Definition

Definition (Database, Support & Occupancy)

Let $\mathcal{D} = \{t_1, \ldots, t_n\}$ be a database over item alphabet $\mathcal{I}$. For an itemset $X \subseteq \mathcal{I}$:

- **Supporting Transactions:** $T(X) = \{t_q \in \mathcal{D} \mid X \subseteq t_q\}$, and $\text{supp}(X) = |T(X)|$.
- **Transaction Occupancy:** $\text{occ}(X, t_q) = \frac{|X|}{|t_q|}$ if $t_q \in T(X)$, else 0.
- **Global Occupancy:** Average coverage across the entire database:

$$\text{occ}(X) \triangleq \frac{1}{|\mathcal{D}|} \sum_{t_q \in T(X)} \frac{|X|}{|t_q|} = \frac{|X|}{|\mathcal{D}|} \sum_{t_q \in T(X)} \frac{1}{|t_q|}$$

The HOI Mining Problem

Given database $\mathcal{D}$ and threshold $\alpha \in (0, 1]$, discover the complete set of High-Occupancy Itemsets:

$$\mathcal{H} \triangleq \{X \subseteq \mathcal{I} \mid \text{occ}(X) \geq \alpha\}$$

The Dual Non-Monotonicity Dilemma

Anti-Monotonicity Fails for Occupancy

In FIM, if $X \subseteq Y$, then $\text{supp}(X) \geq \text{supp}(Y)$ (Apriori downward-closure property). **For occupancy, this monotonicity strictly fails!**

Consider appending an item e to X, yielding $Y = X \cup \{e\}$:

- **Numerator Effect ($\uparrow$):** Cardinality increases from $|X|$ to $|X| + 1$, increasing the fractional factor $\frac{|X|+1}{|t_q|}$.
- **Denominator Effect ($\downarrow$):** Supporting transaction set shrinks: $T(Y) \subseteq T(X)$.

Concrete Divergence

- $X = \{A\}$ in $t_1 = \{A, B, C\}$ ($|t_1| = 3$): $\text{occ}(X) = \frac{1}{3} \approx 0.333$.
- $Y = \{A, B\}$ in $t_1 = \{A, B, C\}$: $\text{occ}(Y) = \frac{2}{3} \approx 0.667 > \text{occ}(X)$! (Superset has **higher** occupancy).
- $Z = \{A, B, D\}$ in t_1: $t_1 \notin T(Z) \implies \text{occ}(Z) = 0$ (Drop to zero).

$\implies$ **Pruning requires sound, tight mathematical upper bounds over all potential descendants.**

State of the Art: The Transaction-Length Bound (UBO)

Existing exact miners (HEP [Deng, 2020] and DFHOI [Nguyen et al., 2023]) establish an upper bound by assuming every descendant $Y \supseteq X$ can grow up to the entire size of each supporting transaction:

Transaction-Length Upper Bound (UBO)

$$\text{occ}(Y, t_q) \leq \frac{|t_q|}{|t_q|} = 1 \implies \text{UBO}(X) \triangleq \frac{1}{|\mathcal{D}|} \sum_{t_q \in T(X)} \frac{|t_q|}{|t_q|} = \frac{|T(X)|}{|\mathcal{D}|} = \frac{\text{supp}(X)}{|\mathcal{D}|}$$

Critical Vulnerability on Dense Databases

- In dense databases (e.g., `mushroom`, `accidents`, `chess`, `pumsb`), transactions are long ($|t_q| \in [23, 74]$) and item co-occurrence is very high.
- $\text{UBO}(X)$ becomes equal or nearly equal to relative support.
- **Combinatorial Explosion:** $\text{UBO}(X)$ grossly overestimates occupancy, forcing miners to generate millions of fruitless candidates and perform expensive TID-list intersections.

Key Geometric Insight: Search-Tree Length Capping

- In a depth-first search tree with a fixed item ordering $\prec$, node X is extended only by items from its candidate tail:

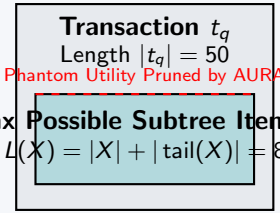
$$\text{tail}(X) = \{e \in \mathcal{I} \mid e \succ \max(X) \text{ and } \text{supp}(X \cup \{e\}) \geq 1$$

- Any descendant $Y \in \text{Subtree}(X)$ can contain **at most** all items of X plus all items in $\text{tail}(X)$:

$$|Y| \leq L(X) \triangleq |X| + |\text{tail}(X)|$$

- **Key Observation on Dense Data:** Deep in the search tree, $|\text{tail}(X)|$ shrinks rapidly, causing $L(X) \ll |t_q|$ for almost all supporting transactions!

The Structural Ceiling



The Length-Capped Residual Bound: Formulation

Definition (Length-Capped Residual Bound $\mathrm{UB}_{\mathrm{res}}(X)$)

Let X be an itemset in the DFS tree with supporting transaction set $T(X)$ and maximum reachable descendant cardinality $L(X) = |X| + |\mathrm{tail}(X)|$. The length-capped residual bound is:

$$\mathrm{UB}_{\mathrm{res}}(X) \triangleq \frac{1}{|\mathcal{D}|} \sum_{t_q \in T(X)} \min\left(1, \frac{L(X)}{|t_q|}\right)$$

Decomposition into Two Regimes

For each supporting transaction $t_q \in T(X)$:

1. If $|t_q| \leq L(X)$: The transaction is short; occupancy could theoretically reach 1. Contribution: 1.
2. If $|t_q| > L(X)$: The transaction is long; occupancy can **never** exceed $\frac{L(X)}{|t_q|} < 1$. Contribution: $\frac{L(X)}{|t_q|}$.

$$\implies \mathrm{UB}_{\mathrm{res}}(X) = \frac{1}{|\mathcal{D}|}\left(\sum_{t_q \in T(X),\, |t_q| \leq L(X)} 1 + L(X) \sum_{t_q \in T(X),\, |t_q| > L(X)} \frac{1}{|t_q|}\right)$$

Theoretical Foundation: Soundness & Dominance

Lemma 1 (Soundness / Upper Bound Guarantee)

For every itemset X and every descendant $Y \in \text{Subtree}(X)$, $\text{occ}(Y) \leq \text{UB}_{\text{res}}(X)$.

Proof Sketch.

Let $Y \in \text{Subtree}(X)$. By definition of the DFS tail, $Y \subseteq X \cup \text{tail}(X)$, so $|Y| \leq L(X)$. Also, $T(Y) \subseteq T(X)$. For every $t_q \in T(Y)$, $\text{occ}(Y, t_q) = \frac{|Y|}{|t_q|} \leq \frac{L(X)}{|t_q|}$, and trivially $\text{occ}(Y, t_q) \leq 1$. Thus, $\text{occ}(Y, t_q) \leq \min\left(1, \frac{L(X)}{|t_q|}\right)$. Summing over $T(Y) \subseteq T(X)$ and dividing by $|\mathcal{D}|$ yields $\text{occ}(Y) \leq \text{UB}_{\text{res}}(X)$. □

Lemma 2 (Strict Dominance over UBO)

For all itemsets X, $\text{UB}_{\text{res}}(X) \leq \text{UBO}(X)$. Furthermore, if $\exists t_q \in T(X)$ such that $|t_q| > L(X)$, then the inequality is **strictly tighter**: $\text{UB}_{\text{res}}(X) < \text{UBO}(X)$.

Proof.

Characterization Theorem: Pruning Scope

Theorem (Exact Scope of Dominance)

$\mathrm{UB}_{res}(X) = \mathrm{UBO}(X)$ *if and only if for all supporting transactions* $t_q \in T(X)$, $|t_q| \leq L(X)$.

Practical Consequence

- **Dense Databases:** Transactions have large lengths (e.g., $|t_q| \approx 30$–75). At moderate depth in the search tree, $|\,\mathrm{tail}(X)|$ falls below 20, making $|t_q| > L(X)$ hold for **nearly 100%** of supporting transactions. $\implies$ Massive, exponential pruning!
- **Sparse Databases:** Transactions are short ($|t_q| \approx 8$–10), while alphabet $\mathcal{I}$ is large ($|\,\mathrm{tail}(X)| \approx 1000$). Here $L(X) \gg |t_q|$, so $\mathrm{UB}_{\mathrm{res}}(X)$ smoothly reduces to $\mathrm{UBO}(X)$ without correctness penalty.

No False Dismissals

Because $\mathrm{UB}_{\mathrm{res}}(X)$ is a mathematically proven upper bound for all descendants, pruning any branch where $\mathrm{UB}_{\mathrm{res}}(X) < \alpha$ guarantees **zero false dismissals** (100% exactness).

Optimization: Uniform-Length Closed Form

[$O(1)$ Bound Evaluation on Uniform Databases] If all transactions in $\mathcal{D}$ have the identical length M (as in standard benchmarks `mushroom` ($M = 23$), `chess` ($M = 37$), and `pumsb` ($M = 74$)), then for any itemset X:

$$\mathrm{UB}_{\mathrm{res}}(X) = \min\left(1, \frac{L(X)}{M}\right) \cdot \frac{\mathrm{supp}(X)}{|\mathcal{D}|}$$

Computational Significance

- Completely eliminates the $O(|T(X)|)$ iteration loop over supporting transactions.
- Evaluated in $O(1)$ arithmetic operations prior to performing candidate intersections!
- If $\min\left(1, \frac{L(X)}{M}\right) \cdot \mathrm{supp}(X) < \alpha \cdot |\mathcal{D}|$, the entire subtree is pruned **instantaneously**.

The AURA-HOI Algorithm: DFS with Equivalence Classes

Algorithm 1 AURA-HOI Recursive Mining Procedure

procedure MINESUBTREE(X, Candidates, α, $\mathcal{D}$)
  $L \leftarrow |X| + |\text{Candidates}|$
  ▷ **Rule 1: Length-Capped Residual Bound Check**
  if $\text{UB}_{\text{res}}(X, L, \mathcal{D}) < \alpha$ **then return**
  end if
  for all $i \in$ Candidates **do**
    $Y \leftarrow X \cup \{i\}$; $T(Y) \leftarrow T(X) \cap T(i)$ ▷ Bitset/TID-list Intersection
    ▷ **Rule 2: Transaction-Length / Support Check**
    if $\text{supp}(Y)/|\mathcal{D}| < \alpha$ **then continue**
    end if
    ▷ **Exact Evaluation & Emission**
    if $\text{occ}(Y) \geq \alpha$ **then**
      output Y
    end if
    $\text{NewCand} \leftarrow \{j \in \text{Candidates} \mid j \succ i \text{ and } T(Y) \cap T(j) \neq \emptyset\}$
    MINESUBTREE(Y, NewCand, α, $\mathcal{D}$)

Exact Rational Arithmetic Kernel

The Peril of IEEE 754 Floating-Point Comparison

- Accumulating thousands of terms $\frac{|X|}{|t_q|}$ introduces numerical drift: $(1/3 + 1/3 + 1/3) \neq 1.0$.
- Near boundary cases ($\text{occ}(X) \approx \alpha$), miners using floating-point can either spuriously discard true HOIs or admit invalid patterns.

Exact Rational Decision in AURA-HOI (`dm_exact_occ.h`)

- When α is rational $\frac{p}{q}$, the threshold condition $\frac{|X|}{|\mathcal{D}|} \sum_q \frac{1}{|t_q|} \geq \frac{p}{q}$ is rewritten as:

$$q \cdot |X| \cdot \sum_{t_q \in T(X)} \frac{\text{LCM}(\{|t_q|\})}{|t_q|} \geq p \cdot |\mathcal{D}| \cdot \text{LCM}(\{|t_q|\})$$

- Fast-path double comparison outside an exact ϵ-tolerance band; switches to exact bignum rational integer comparison inside the band.
- **Guarantees 100% mathematical parity and bit-exact reproducibility.**

Chunked Arena Memory Architecture

Problem with Dynamic Allocations

- Classical miners allocate TID-lists via individual `malloc()`/`free()` calls or Java objects.
- In dense trees with millions of nodes, this causes:
  - Catastrophic heap fragmentation.
  - High CPU overhead in allocator locks.
  - Poor cache locality.

AURA-HOI Chunked Arena Allocator

- Pre-allocates large, contiguous memory slabs (chunks of 4–16 MiB).
- Allocation is a pointer bump:

$$\text{ptr} = \text{arena.top}; \quad \text{arena.top} += \text{size};$$

- Recursion unwind: Entire tree level released instantaneously by restoring top pointer.
- **Yields up to 22% runtime reduction on dense datasets!**

Experimental Protocol & Benchmark Datasets

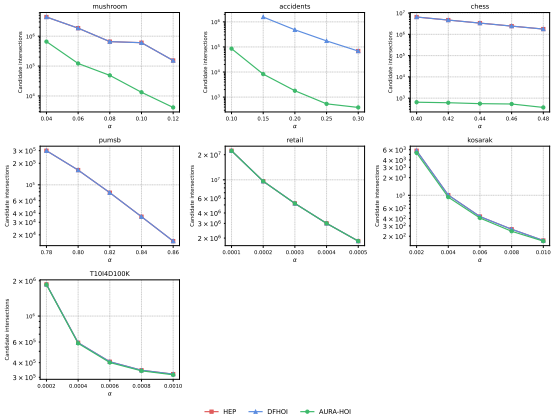
Standard Benchmark Datasets (SPMF/FIMI)

Dataset	Type	n	$\lvert\mathcal{I}\rvert$	Avg. Len	Density
`mushroom`	Dense	8,416	119	23.0	19.33%
`accidents`	Dense	340,183	468	33.8	7.22%
`chess`	Dense	3,196	75	37.0	49.33%
`pumsb`	Dense	49,046	2,113	74.0	3.50%
`retail`	Sparse	88,162	16,470	10.3	0.06%
`kosarak`	Sparse	990,002	41,270	8.1	0.02%
`T10I4D100K`	Sparse	100,000	870	10.1	1.16%

Rigorous Setup

- **Hardware:** Intel Core i7-10700 (2.9–4.8 GHz), 32 GiB DDR4 RAM, Ubuntu 24.04 LTS.
- **Baselines:** HEP [Deng 2020] and DFHOI [Nguyen 2023], both implemented in optimized native C.
- **Execution Caps:** 600 s wall-clock time limit (TL), 20 GiB address space limit (ML).
- Single thread pinned with `taskset`.

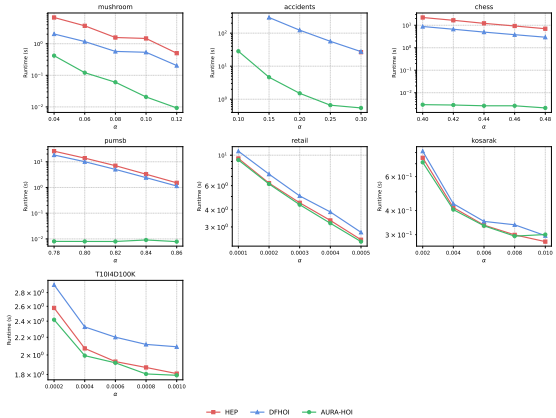
RQ1: Pruning Efficacy (Candidate Intersections)



Language-Independent Proof

- Number of candidate intersections is an invariant metric of pure mathematical pruning tightness.
- **Dense Results:**
  - `mushroom`: **99.4%** node reduction over DFHOI.
  - `accidents`: **99.5%** node reduction!
  - `chess`: reduces intersections from 3.37×10^6 to 551 ($> 99.98\%$).
  - `pumsb`: reduces from 7.7×10^4 to 0.
- Eliminates virtually all combinatorial exploration in dense search spaces!

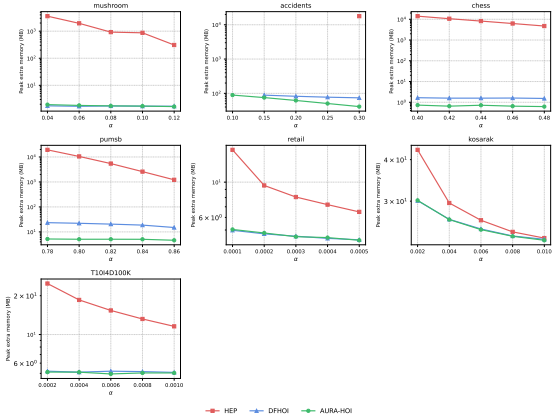
RQ2: Runtime Efficiency



Speedup Breakdown

- **On Dense Datasets:**
  - `mushroom`: AURA-HOI is 12× to 26× faster than DFHOI.
  - `accidents`: 69× to 85.7× faster!
  - `chess` & `pumsb`: $1,400\times$ **to** $2,989\times$ faster than DFHOI, up to $7,518\times$ faster than HEP!
- **Overcoming Baseline Failure:** At $\alpha = 0.10$ on `accidents`, DFHOI times out (> 600 s) and HEP crashes (> 20 GiB RAM). AURA-HOI finishes in **17.8 seconds**!
- **Sparse Datasets:** Speedup ratio remains 1.02×–1.21×, confirming no performance overhead.

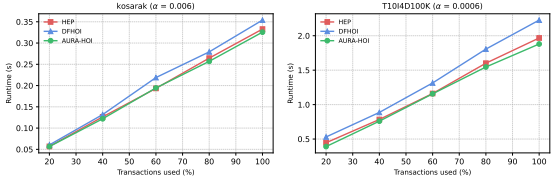
RQ3: Peak Memory Footprint



Memory Stability

- HEP experiences explosive memory growth due to maintaining large candidate structures ($>$ 19.2 GiB on `accidents`).
- DFHOI requires up to 80 MB.
- AURA-HOI peak memory is strictly contained between **5 MB and 89 MB** across all configurations.
- Chunked arena allocator ensures zero memory leaks and predictable, rock-solid heap footprint.

RQ4: Scalability Analysis



Linear Scalability with Database Size

- Tested on prefix slices: 20%, 40%, 60%, 80%, 100% of transactions on `kosarak` and `T10I4D100K`.
- Runtimes grow in direct linear proportion to transaction volume.
- Relative ordering ($\text{AURA-HOI} \approx \text{DFHOI} \leq \text{HEP}$) remains completely invariant across all data scales.

Ablation Study: Dissecting Pruning Components

Dataset	α	Full AURA-HOI	No Tail Cap	No Class Bound	No Residual	No Arena
`mushroom`	0.08	**0.059 s**	0.062 s	0.082 s	0.425 s	0.076 s
(Intersections)		48,890	50,905	75,947	657,947	48,890
`accidents`	0.20	**1.47 s**	1.50 s	3.47 s	91.7 s	1.54 s
(Intersections)		1,794	1,839	6,539	482,214	1,794

Ablation Insights

- **No Residual (UBO only):** Intersections surge by 13.4× on `mushroom` and 268× on `accidents`. Runtime increases by 7.2× and 62×!
- **No Class Bound:** Intersections increase by 55% on `mushroom` and 3.6× on `accidents`.
- **Arena Allocator:** Grants a pure 22% speedup on `mushroom` by eliminating allocation overhead.

Summary of Key Contributions

1. **Theoretical Breakthrough:** Formulated the **Length-Capped Residual Bound** ($\mathrm{UB}_{\mathrm{res}}$), proving that search-tree tail limits restrict the maximum achievable occupancy on long transactions.
2. **Strict Dominance:** Mathematically proved that $\mathrm{UB}_{\mathrm{res}}(X) \leq \mathrm{UBO}(X)$ universally, with strict inequality on all dense databases.
3. **Closed-Form Evaluation:** Established an $O(1)$ uniform-length closed form that slashes bound evaluation cost to zero.
4. **High-Performance Architecture:** Developed an exact rational arithmetic kernel and a chunked arena allocator in native C.
5. **Empirical Superiority:** Demonstrated up to **99.5%** candidate intersection reduction, up to $2,989\times$ speedup over DFHOI, and successfully mined difficult thresholds where baselines failed.

Future Directions & Discussion

Open Avenues for Research

- **Maximal & Closed HOIs (MFHOI):** Extending length-capped residual bounds to mine only Pareto-maximal high-occupancy itemsets without redundancy.
- **Per-Transaction Residual Bounds for Sparse Data:** Developing hybrid bounds factoring in transaction-specific item distributions to enhance sparse pruning.
- **Downstream Integration:** Leveraging exact HOIs as compact knowledge primitives for neuro-symbolic recommender systems and clinical pathway discovery.

Thank You for Your Attention!

Q&A / Open Discussion